

RECURSION DEEP DIVE

"Three patterns, one skeleton, and a call stack you can finally see."



■ WHERE WE ARE — AND WHY TODAY MATTERS ■

Last class you typed `factorial(n)` once. It worked, but it probably still felt weird. Today we go deep. Why? Because every tree, BST, and heap class for the next 5 sessions is recursive. If recursion clicks today, the rest is smooth.

Block	What we cover
1	The call stack — see what really happens when a function calls itself
2	The three recursive patterns — decrement, walk, divide
3	The four classical bugs and how to spot them
4	Practice — 3 warm-ups, recursion on nodes, recursive binary search

"If recursion clicks today, the rest of the semester is downhill."

■ PATTERN 1 — DECREMENT ■

The input shrinks by one each call. Goes down to 0 or 1. The simplest recursive shape — the one you saw in **factorial**. Use it whenever the problem is "do something, then do the same on $n-1$ ".

```
def count_down(n):  
    if n < 0:  
        return # base case  
  
    print(n) # do something with current  
    count_down(n - 1) # shrink by 1
```

```
def factorial(n):  
    if n == 0:  
        return 1  
  
    return n * factorial(n - 1)  
  
# shrink by 1, combine with *
```

"I handle the current step. I pass the rest, slightly smaller, to a smaller me."

■ PATTERN 1 — DECREMENT SKELETON ■

Every recursive function in this course follows the same shape. Three lines. Three questions. **Memorize it now** and the rest of the semester is muscle memory.

```
def f(input):  
    if <base case>: # 1) stop?  
        return <neutral value> # 2) return?  
    return <combine>(<curr>, f(<smaller>))  
    # 3) shrink the problem
```

[01] When do we stop?

For `factorial`: when `n == 0`. Always answer this first.

[02] What do we return at the bottom?

For `factorial`: `1` — the neutral element of multiplication.

[03] How do we shrink?

For `factorial`: call `f(n-1)` and multiply by `n`.

"Write the base case FIRST. Always. It is your safety net against `RecursionError`."

■ PATTERN 2 — WALK ■

The input is now a STRUCTURE — a Python list, or a chain of linked nodes. At each call, you process ONE element (the head) and recurse on what remains (the rest). This is the pattern that comes back on every tree operation from class 05 on. Drill it today.

```
def sum_list(lst):  
    if not lst: # base case: empty list  
        return 0  
  
    return lst[0] + sum_list(lst[1:]) # head + rest
```

```
def length_recursive(node): # walks a linked list  
    if node is None: # end of chain  
        return 0  
  
    return 1 + length_recursive(node.next)
```

Same skeleton, two vehicles.

sum_list walks a Python list : base = not lst, shrink = lst[1:].

length_recursive walks linked nodes : base = node is None, shrink = node.next.

"Walk the structure. One step at a time. Combine the current with the rest."

■ PATTERN 2 — WALK SKELETON ■

Same exercise as before — strip the meaning, keep the shape. The walk skeleton has the same three slots as decrement, but the "smaller input" is now "the rest of the structure" instead of $n - 1$. Once you internalize this template, you can walk any list, any chain, any tree.

```
def f(structure):  
    if <structure is empty>: # 1) base case  
        return <neutral value> # 2) bottom-of-stack  
    return <combine>(<head>, f(<rest>)) # 3) head +  
    rest
```

[01] On a Python list

```
empty = not lst.head = lst[0].rest = lst[1:].
```

[02] On a linked node chain

```
empty = node is None. head = node.data. rest =  
node.next.
```

[03] On a tree (next class)

empty = node is None. rest = TWO recursive calls. **Same skeleton. Just one more recursive call per level.**

"Head, then rest. Lists, chains, trees — same skeleton, different rest."

■ PATTERN 3 — DIVIDE ■

At each step you split the problem and keep going on only ONE half. The classic example is binary search on a sorted list. After about $\log_2(n)$ calls, the search window is empty. That is where $O(\log n)$ is born — not by walking the whole list, but by halving it each time.

[01] What "divide" means

You split the input in half, identify which half can still contain the answer, and recurse on JUST that half. The other half is gone forever — you never look at it again.

[02] Where it shows up later

Binary search (today's stretch). BST search (class 07) — same shape, on a tree. Merge sort and quick sort (classes 15–16) — divide a list, recurse on each half, then combine.

[03] Why $O(\log n)$

The window halves at each call. Starting from n , you reach an empty window after about $\log_2(n)$ halvings. For $n = 1,000,000$ that is ~20 calls instead of one million.

*"Split. Keep one half. Forget the other.
That is how $O(\log n)$ is born."*

■ DIVIDE — A FULL WORKED EXAMPLE ■

Recursive binary search, written out in full. A sorted list, a target, a window defined by **low** and **high**. At each call, look at the middle. Either you found the target, or you know which half cannot possibly contain it — and you throw that half away. The function calls itself on the surviving half only.

```
def binary_search_rec(lst, target, low=None, high=None):  
    if low is None: # first call only  
        low, high = 0, len(lst) - 1  
    if low > high: # base case: empty window  
        return -1  
    mid = (low + high) // 2  
    if lst[mid] == target:  
        return mid # found  
    if target < lst[mid]:  
        return binary_search_rec(lst, target, low, mid - 1) #  
        left  
    return binary_search_rec(lst, target, mid + 1, high) # right
```

[01] Trace — `binary_search_rec([1, 3, 5, 7, 9], 7)`

Call 1 → low=0, high=4, mid=2, lst[2]=5. 5 < 7 → recurse RIGHT.

Call 2 → low=3, high=4, mid=3, lst[3]=7. Found → return 3.

Two calls. $\log_2(5) \approx 2.3$. Maths confirmed.

[02] Why low=None, not low=0

The Python idiom for defaults: declare as None, then set inside on first call. Forces value to be computed fresh every time.

[03] Why each call halves the window

mid-1 and mid+1 exclude the cell just checked. The next window is strictly smaller, by half. After $\sim \log_2(n)$ calls, low > high and we stop.

"Three slots, two halves, one half discarded forever — that is divide in code."

■ PATTERN 3 — DIVIDE SKELETON ■

Strip the binary search and look at the shape. Divide differs from decrement and walk on one point: there are **TWO possible recursive paths** (left half or right half), but only **ONE is taken** at each call. The other half is gone for good. The skeleton encodes that choice.

```
def f(window):  
    if <window is empty>: # 1) base  
        return <not-found value>  
    mid = <middle of window>  
    if <found at mid>:  
        return mid # hit  
    if <target lives in left half>:  
        return f(<left half>) # ONE half  
    return f(<right half>) # otherwise
```

[01] What shrinks

The window. Not by 1 (decrement), not by "head removed" (walk) — by HALF. That is what gives $O(\log n)$ instead of $O(n)$.

[02] Why only one recursive call fires

Once you know the target cannot be in one half, you ignore it entirely. You never recurse on both. (Sorts will recurse on both).

[03] Same shape on a tree

Replace `window` with `subtree`, `mid` with `node`. That is BST search — class 07. Today's pattern, applied to a tree.

"Same skeleton — base case, recursive call on a smaller piece — but the piece is half, and there are two halves to choose from."

■ PRACTICE — CORE + STRETCH ■

Practice today is split into Core (mandatory, prepares the next class) and Stretch (bonus, for those who finish early). Do Core in order — each block builds the reflex you need for trees on Monday.

[01] 🎯 Core 1 — Three warm-ups

`repeat(message, n)`, `product_list(lst)`, `digits_sum(n)`.
Three short functions to drill *base case + recursive case* three times in a row. Patterns: decrement (side-effect), walk on a Python list, decrement with `n // 10` shrink.

[02] 🎯 Core 2 — Recursion on nodes

`contains_in_chain(node, target)` and `max_in_chain(node)` on a chain of `_Node` objects. Same walk pattern that will work on trees with `node.left` and `node.right` next class.

[03] ★ Stretch — `integer_sqrt`

Apply class 01's binary search pattern to a new problem — find the floor of $\sqrt{n}$ in $O(\log n)$. Same algorithm shape, different domain: you binary-search the answer range `[0, n]` instead of a sorted list. Bonus only; class 07 will revisit divide-recursion in detail on trees.